INFORME TÉCNICO DEL PROYECTO

Tucumán Respira

Panel web de monitoreo de calidad del aire IoT y firmwares de la estación (Arduino Uno + ESP32)

Aspecto	Detalle
Encargado	Benicio Gael Pieroni
Proyecto	Tucumán Respira — Estación de monitoreo ambiental IoT
Entregables	Aplicación web estática (index.html · styles.css · app.js) y dos firmwares: Arduino Uno (sketch_oct17a) y ESP32 (sketch_oct17b)
Fuente de datos	Supabase (PostgreSQL) — tabla «mediciones» (API PostgREST)
Versión del informe	v2 — amplía el informe original del panel web con los firmwares, la integración y las últimas correcciones
Fecha del informe	18/10/2026
Estado	Implementado, integrado y verificado (pendiente de publicación online)

Este documento describe el diseño, la arquitectura, las funcionalidades, los umbrales de referencia, los firmwares, las correcciones aplicadas y las verificaciones realizadas durante el desarrollo del sistema «Tucumán Respira». La versión 2 incorpora el firmware del Arduino Uno (adquisición y GPS), el firmware del ESP32 (WiFi, carga a Supabase y alarma sonora) y el control del zumbador desde el panel web.

Índice de contenidos

1. Resumen ejecutivo

2. Contexto y objetivos del proyecto

3. Decisiones de diseño acordadas

4. Arquitectura y stack tecnológico

5. Estructura de archivos y métricas de código

6. Fuente de datos: Supabase

7. Métricas y umbrales de referencia

8. Índice de Calidad del Aire (ICA / EPA)

9. Funcionalidades del panel web

10. Estados del sistema y comportamiento «offline»

11. Adaptación responsive (móviles y tablets)

12. Seguridad

13. Firmware del Arduino Uno (adquisición y GPS)

14. Firmware del ESP32 (WiFi, Supabase y alarma)

15. Integración del sistema y control del zumbador

16. Correcciones de errores durante el desarrollo

17. Verificaciones y pruebas realizadas

18. Cómo ejecutar y visualizar el sistema

19. Pendientes y recomendaciones

Anexo A — Configuración y claves

Anexo B — Exportación de datos (CSV / JSON)

Anexo C — Trama de comunicación Uno → ESP32

Anexo D — Prompt para recrear el proyecto con opencode

1. Resumen ejecutivo

«Tucumán Respira» es una estación de monitoreo ambiental que mide la calidad del aire y variables meteorológicas y publica sus lecturas en la nube. El sistema se compone de cuatro piezas: un **Arduino Uno** que adquiere los sensores y calcula la posición GPS; un **ESP32** que se conecta a Wi-Fi, envía las mediciones a **Supabase** y gobierna una alarma sonora con zumbador; una **base de datos PostgreSQL** en Supabase; y una **aplicación web estática** que muestra los datos en vivo y permite descargarlos y detener la alarma de forma remota.

La versión 2 del informe documenta tanto el panel web (rediseñado por completo) como los dos firmwares y su integración, e incorpora las últimas correcciones: el mapa se renderiza con OpenStreetMap (sin clave de API), la descarga JSON se entrega sin truncar, la app muestra la última coordenada GPS válida cuando la estación pierde señal, y el zumbador puede detenerse desde la web sin necesidad de cargar ninguna dirección IP.

Resultados principales

• Panel web estático (HTML + CSS + JS) con datos en vivo desde Supabase y respaldo por sondeo cada 15 s.

• 8 métricas con umbrales por estándar (EPA, OMS/ICNIRP, OSHA/NIOSH y ASHRAE), cada una con su fuente visible.

• Índice de calidad del aire (ICA/EPA) calculado con PM2.5 y CO.

• Histórico por sensor, gráfico de tendencia, y descarga pública en CSV y JSON (100, 500, 1000 o todas las filas).

• Mapa de ubicación con OpenStreetMap y marcador de la estación; usa la última coordenada GPS válida.

• Botón «Detener zumbador» en la web: la dirección IP de la estación se obtiene sola desde Supabase.

• Firmware del Arduino Uno: 8 sensores, GPS NMEA y trama de 9 campos hacia el ESP32.

• Firmware del ESP32: Wi-Fi (modo estación + punto de acceso de configuración), carga a Supabase, alarma por zumbador y endpoints de control (/status, /alarma, /silenciar).

Estado actual verificado en la base de datos al 18/10/2026: **1.284 lecturas**, la última el 18/10/2026 a las 15:50:53 UTC. De ellas, 533 incluyen coordenadas GPS y 751 son anteriores al soporte de GPS (sin latitud/longitud). La comunicación en vivo funciona (última lectura hace minutos).

2. Contexto y objetivos del proyecto

El proyecto «Tucumán Respira» surge con la finalidad de acercar a la comunidad datos ambientales reales de su entorno, con foco en colegios, estudiantes y docentes de la Provincia de Tucumán. La estación funciona de forma autónoma y publica sus mediciones para que cualquiera pueda consultarlas sin instalar nada.

Objetivos

• Medir en forma continua variables de calidad del aire (PM2.5, CO■, CO), meteorológicas (temperatura, humedad, presión), radiación UV y luz ambiental.

• Registrar la ubicación geográfica (GPS) de las mediciones.

• Publicar los datos en la nube y ofrecer un panel público que los muestre de forma clara.

• Avisar localmente (alarma sonora) cuando la calidad del aire supera umbrales peligrosos, y permitir silenciar esa alarma desde la aplicación web.

• Permitir la descarga libre de los datos crudos (CSV/JSON).

Alcance de esta versión del informe

La versión 2 amplía el informe original (centrado en el panel web) para cubrir también el firmware del Arduino Uno, el firmware del ESP32 y la integración entre ambos, la base de datos y el control remoto del zumbador.

3. Decisiones de diseño acordadas

Tema	Decisión adoptada
Zumbador	Ubicado y gobernado por el ESP32 (pin GPIO 23); la condición de alarma la decide la propia ESP32.
Alarma	Se activa al superar ICA > 100 y se corta al bajar de 80; el silencio manual dura 10 minutos.
GPS	Conectado al Arduino Uno en los pines 7 (RX) y 8 (TX) por SoftwareSerial.
Luz	Un único módulo LDR («MH-MQ»), sin agregar columnas nuevas a la base.
Detener alarma	Botón directo en la app que consulta a la ESP32 por HTTP; la IP se obtiene automáticamente desde Supabase.
Base de datos	No se cambian los campos existentes; solo se agrega la columna «ip_dispositivo».
Imágenes de mapa	OpenStreetMap (sin clave de API).
Idioma / estilo	Español; código sin comentarios añadidos; autor del informe: Benicio Gael Pieroni.

4. Arquitectura y stack tecnológico

La arquitectura es de cuatro capas con comunicación en cascada:

• **Adquisición (Arduino Uno):** lee los sensores y el GPS y envía una trama de 9 campos por puerto serie al ESP32.

• **Enlace y nube (ESP32):** se conecta al Wi-Fi del lugar, expone un punto de acceso de configuración y realiza peticiones HTTPS a la API REST de Supabase (PostgREST).

• **Persistencia (Supabase):** base PostgreSQL con la tabla «mediciones».

• **Presentación (app web):** aplicación estática que lee Supabase y, para la alarma, consulta directamente al ESP32 en la red local.

Capa	Tecnología	Rol
Adquisición	Arduino Uno (ATmega328P)	Lectura de sensores y GPS; trama serie.
Enlace	ESP32 (Wi-Fi + HTTPS)	Carga a la nube, alarma y portal de configuración.
Datos	Supabase / PostgreSQL / PostgREST	Almacenamiento y API REST.
Presentación	HTML5 + CSS3 + JavaScript (Leaflet, Chart.js, Lucide)	Panel público y descargas.

5. Estructura de archivos y métricas de código

El proyecto se organiza en dos carpetas: la aplicación web («App-Web2») y los firmwares («Firmwares_Tucuman_Respira»).

Archivo	Rol	Líneas aprox.
index.html	Estructura y semántica de la interfaz; carga de librerías y de app.js.	271
styles.css	Sistema de diseño: tokens de tema, componentes y reglas responsive.	678
app.js	Configuración, datos, clasificación, gráficos, mapa, modales y descargas.	1012
sketch_oct17a.ino	Firmware del Arduino Uno (adquisición, GPS y trama serie).	274
sketch_oct17b.ino	Firmware del ESP32 (WiFi, Supabase, alarma y portal).	842
Total	—	≈ 3.077

Organización interna de app.js

• CONFIG: parámetros centrales (URL y clave de Supabase, tabla, límites, tiempos, clave del dispositivo, IP por defecto).

• METRICS: definición de los 8 sensores, unidades, decimales, umbrales y fuentes.

• TREND_TABS y AQI_*: pestañas del gráfico de tendencia y tablas del índice de calidad del aire.

• Utilidades: formato de fecha/hora, tiempo relativo, formateo, escapado de CSV y validación de GPS.

• ICA (EPA): cálculo del subíndice y del índice global.

• Render: tarjetas, banner, resumen, reloj de estado, mapa, gráficos, modales y descargas.

• Alarma: función que obtiene la IP del dispositivo desde Supabase y llama a su endpoint de silencio.

6. Fuente de datos: Supabase

La información proviene de un proyecto de Supabase (PostgreSQL) con la API PostgREST habilitada. La tabla utilizada es «mediciones».

Campo	Tipo	Significado	Unidad
id	entero	Identificador autoincremental de la fila.	—
created_at	timestamp	Fecha y hora de la lectura (UTC).	ISO-8601
co2	numérico	Dióxido de carbono.	ppm
co	numérico	Monóxido de carbono.	ppm
pm25	numérico	Material particulado fino (PM2.5).	µg/m³
temperatura	numérico	Temperatura ambiente.	°C
humedad	numérico	Humedad relativa.	%
presion	numérico	Presión atmosférica.	hPa
uv	numérico	Índice de radiación ultravioleta.	índice
luz	numérico	Luminosidad ambiental.	lux
latitud	numérico	Latitud GPS de la estación.	grados
longitud	numérico	Longitud GPS de la estación.	grados
ip_dispositivo	texto	IP local actual del ESP32 (nueva columna).	IPv4

Estado verificado de la base de datos (al 18/10/2026)

Métrica	Valor
Filas totales	1.284
Primera lectura	05/10/2026 00:45:18 UTC
Última lectura	18/10/2026 15:50:53 UTC
Lecturas con coordenadas	533
Lecturas sin coordenadas (firmware previo sin GPS)	751
Lecturas con «ip_dispositivo»	117 (desde que se activó la publicación de la IP)

La columna «ip_dispositivo» permite que la aplicación descubra sola la dirección de la estación en la red local y, con ella, detener el zumbador sin que el usuario tenga que escribir ninguna IP.

7. Métricas y umbrales de referencia

Cada sensor se clasifica según rangos de referencia tomados de fuentes reconocidas de salud y seguridad. Todas las métricas cuentan con niveles que llegan a «peligro», con su fuente visible en la interfaz. El estado general (ICA) se calcula con PM2.5 y CO; el resto de las métricas marcan su propio nivel de alerta en su tarjeta.

Métrica	Unidad	Umbrales y etiquetas	Fuente
CO■	ppm	Bueno ≤800 · Aceptable ≤1000 · Ventilación pobre ≤1500 · Alta (PEL 8 h) ≤5000 · Muy alta (STEL) ≤30000 · Peligro >30000	OSHA/NIOSH · ASHRAE
CO (ICA)	ppm	Bueno ≤4,4 · Moderado ≤9,4 · Grupos sensibles ≤12,4 · Dañino ≤15,4 · Muy dañino ≤30,4 · Peligroso ≤50,4 · Extremo >50,4	EPA AQI (8 h) · OSHA
PM2.5 (ICA)	µg/m³	Bueno ≤9,0 · Moderado ≤35,4 · Grupos sensibles ≤55,4 · Dañino ≤125,4 · Muy dañino ≤225,4 · Peligroso >225,4	EPA AQI 2024 (24 h)
Temperatura	°C	Hipotermia ≤0 · Frío ≤10 · Confortable ≤27 · Precaución por calor ≤32 · Extrema precaución ≤39 · Peligro ≤52 · Peligro extremo >52	NWS/OSHA · NIOSH
Humedad	%	Muy seco ≤20 · Seco ≤30 · Ideal ≤60 · Algo húmedo ≤70 · Riesgo de moho ≤85 · Alto riesgo de moho >85	OMS · ASHRAE
Presión	hPa	Baja ≤1004,9 · Normal ≤1020 · Alta >1020	Informativo
UV	índice	Bajo ≤2 · Moderado ≤5 · Alto ≤7 · Muy alto ≤10 · Extremo >10	OMS / ICNIRP
Luz	lux	Oscuridad ≤9,9 · Penumbra ≤99,9 · Interior ≤499 · Pleno sol >499	Informativo

Notas

• Los valores son de referencia para interpretar las mediciones y no constituyen diagnóstico sanitario.

• La OMS no fija un umbral cuantitativo de humedad: los niveles altos indican riesgo de moho, no peligro agudo.

• Presión y luz se muestran como referencia: no existe un umbral de peligro para la salud humana en esos rangos.

• El CO■ alto es peligroso solo en niveles ocupacionales; en interiores es ante todo un indicador de ventilación.

8. Índice de Calidad del Aire (ICA / EPA)

El estado general de la calidad del aire se resume con un índice estilo ICA (Air Quality Index de la EPA). Se calcula el subíndice de PM2.5 y el de CO y se toma el peor de los dos como índice global. El mismo cálculo se replica en el firmware del ESP32 para disparar la alarma sonora.

Rango ICA	Categoría	Descripción
0 – 50	Buena	La calidad del aire es satisfactoria y el riesgo para la salud es bajo.

Rango ICA	Categoría	Descripción
51 – 100	Moderada	Calidad aceptable; personas muy sensibles pueden verse afectadas.
101 – 150	Dañina para grupos sensibles	Posibles efectos en personas sensibles.
151 – 200	Dañina	Toda la población puede empezar a notar efectos.
201 – 300	Muy dañina	Riesgo para la salud de toda la población.
301 – 500	Peligrosa	Condiciones de emergencia sanitaria.

El subíndice se obtiene por interpolación lineal dentro del rango correspondiente. La alarma sonora del ESP32 se activa cuando el ICA supera 100 (categoría «Dañina para grupos sensibles» en adelante) y se corta automáticamente cuando baja de 80.

9. Funcionalidades del panel web

9.1 Encabezado y estado

• Marca y conmutador de tema; indicador de conexión (en vivo / retraso / sin conexión).

• Reloj de estado que recalcula el tiempo desde la última lectura.

9.2 Resumen general (ICA) y tarjetas de sensores

• Banner con el ICA, su categoría, descripción y recomendación sanitaria.

• Ocho tarjetas (CO■, CO, PM2.5, temperatura, humedad, presión, UV y luz) con valor, clasificación, barra de nivel y resaltado al alcanzar un nivel peligroso.

9.3 Resumen reciente y gráfico de tendencia

• Mín./máx. de temperatura, pico de PM2.5, promedio de CO■ y de humedad de las últimas 100 lecturas.

• Pestañas Aire, Clima, Gases y Luz/UV con gráfico de líneas interactivo.

9.4 Histórico por sensor (modal)

• Estadísticas (actual, mín., promedio, máx.), selector de 20/50/100 puntos y gráfico del sensor.

9.5 Mapa de ubicación

• Mapa construido con **Leaflet** y mosaicos de **OpenStreetMap** (sin clave de API), con marcador de la estación.

• Muestra la insignia «Ubicación por GPS» y las coordenadas; si la lectura más reciente no tiene posición válida, se usa la **última coordenada GPS válida** conocida y se indica «Última ubicación conocida».

• Incluye un enlace «Ver en Google Maps» cuando hay coordenadas.

9.6 Descarga pública de datos

• Formatos CSV (planilla) y JSON (datos crudos); rangos: últimas 100 (por defecto), 500, 1000 o todas.

• El CSV incluye las columnas latitud y longitud; el JSON incluye todos los campos de cada fila.

• La descarga de «todas» usa paginación por rangos para traer el conjunto completo.

9.7 Control del zumbador desde la app

• Botón «**Detener zumbador**» en el panel de alerta.

• La aplicación obtiene la IP de la estación **automáticamente** desde el último registro de Supabase (campo «ip_dispositivo»); si aún no hay ninguno, usa 192.168.4.1 (punto de acceso de la ESP32).

• El botón envía la orden al endpoint /silenciar del ESP32; la respuesta indica los minutos de silencio.

9.7b Tema y detalles

• Tema claro/oscuro persistente; gráficos y mapa se recolorean al cambiar.

• Aviso automático de estación desconectada, descartable, y actualización en tiempo real + sondeo.

10. Estados del sistema y comportamiento «offline»

Situación	Comportamiento de la interfaz
Cargando	Tarjetas «esqueleto» animadas; indicador «Conectando…».
En vivo (≤120 s)	Indicador verde con pulso; texto «En vivo · hace X».
Retraso (≤300 s)	Indicador ámbar; texto «Retraso · hace X».
Sin conexión (>300 s)	Indicador rojo; aviso con la fecha y hora de la última lectura.
Sin dados	Valores «--» y etiqueta «Sin datos».
Sin GPS	Se usa la última coordenada válida; si no hay ninguna, el mapa se centra por defecto.

Un vigilante («watchdog») actualiza el estado cada segundo y recalcula el tiempo transcurrido respecto de la última lectura, de modo que el mensaje siempre refleja la realidad.

11. Adaptación responsive (móviles y tablets)

Ancho	Cambios principales
≤ 1024 px	Banner y resumen en una columna; sensores en 2 columnas; tendencia y mapa apilados.
≤ 640 px	Marca compacta; una columna de sensores; tipografías y paddings reducidos; gráficos y mapa de 240–260 px; campos de descarga en una columna; modales con menos margen.
≤ 400 px	Botones, tipografías y pestañas más compactos.

• El gráfico del histórico se construye con el modal ya abierto para evitar gráficos colapsados.

• El mapa recalcula su tamaño al girar el dispositivo o redimensionar la ventana.

• El modal de descarga usa columnas minmax(0,1fr) y min-width:0 para no desbordar al abrir los desplegables en móviles, y un ancho que nunca supera la pantalla.

• Se desactivan animaciones si el usuario configuró «reducir movimiento».

12. Seguridad

• Se utiliza la clave pública (anon) de Supabase, diseñada para exponerse en el cliente.

• La aplicación es de solo lectura sobre los datos de la estación.

• La protección real depende de las políticas RLS (Row Level Security) del servidor.

• La configuración se centraliza en el bloque CONFIG de app.js.

• El endpoint «/silenciar» del ESP32 exige una clave compartida («k») para evitar usos no autorizados.

• No se exponen datos sensibles del usuario; la descarga es pública por decisión del proyecto.

13. Firmware del Arduino Uno (adquisición y GPS)

El Arduino Uno actúa como **maestro de adquisición**. Lee los sensores, interpreta las sentencias NMEA del módulo GPS y envía periódicamente una trama de texto al ESP32 por puerto serie.

Sensores y pines

Sensor	Magnitud	Conexión
BME280 (GY-BMEP280)	Temperatura, humedad, presión	I²C (A4/A5), dirección 0x76/0x77
MH-Z19C	CO■ (ppm)	SoftwareSerial RX=10, TX=11
MQ-7	Monóxido de carbono (CO)	Analógico A0
Módulo LDR («MH-MQ»)	Luz ambiental (lux aproximado)	Analógico A1 · digital D12
GY-8511 (ML8511)	Radiación UV	Analógico A2
GY-GPS6MV2 (NEO-6M)	Latitud y longitud	SoftwareSerial RX=7, TX=8
ESP32	Enlace serie de datos	SoftwareSerial RX=2, TX=3

Funcionamiento

• El GPS se deja como receptor activo por defecto (gpsSerial.listen()); se conmuta brevemente al puerto del sensor de CO■ para leerlo y se vuelve al GPS.

• Se interpretan las sentencias **$GPGGA** y **$GNGGA**: se verifica la calidad del fix y se convierten latitud/longitud de formato NMEA (ggmm.mmmm) a grados decimales, aplicando el signo según N/S y E/W.

• El MQ-7 requiere un calentamiento inicial; el sistema espera 30 segundos antes de empezar.

• Las lecturas del BME280 se protegen contra valores inválidos (NaN).

• Cada 3 segundos se envía al ESP32 una trama de 9 campos.

Conservación de la última coordenada válida

Si el GPS pierde el fix momentáneamente, el firmware **no** envía (0,0): conserva la última posición válida y la reutiliza hasta recuperar la señal. Esto evita filas sin coordenadas originadas por cortes transitorios del GPS.

Trama enviada al ESP32

```
co2,co,temperatura,humedad,presion,uv,luz,latitud,longitud

Ejemplo:
412,3.20,19.9,71,968,0.0,120,-26.83972,-65.29328
```

14. Firmware del ESP32 (WiFi, Supabase y alarma)

El ESP32 es el **enlace con la nube** y el **controlador de la alarma**. Recibe la trama del Arduino Uno, arma el registro y lo publica en Supabase; además, mide el ICA y gobierna el zumbador.

Conectividad

• Conexión al Wi-Fi del lugar en modo estación (STA), con reconexión automática.

• Punto de acceso permanente **Tucuman-Respira-Config** (contraseña Tucuman2026, IP 192.168.4.1) con portal DNS cautivo, para configurar la red sin flashear.

• Las credenciales se guardan en memoria no volátil (NVS) con Preferences.

Publicación en Supabase

• Petición HTTPS POST a la API REST de Supabase con encabezados apikey y Authorization (clave anon).

• Campos enviados: co2, co, pm25, temperatura, humedad, presion, uv, luz, latitud, longitud e **ip_dispositivo** (IP local del ESP32, para el descubrimiento automático desde la app).

• Se eliminó el envío de un campo inexistente («aqi») que provocaba un error HTTP 400.

• El puerto del sensor de partículas (PMS5003) se lee por Serial2 (RX 18 / TX 5, SET 4).

Alarma sonora

• Zumbador pasivo en GPIO 23, controlado con tone()/noTone().

• El ICA se calcula localmente con los mismos breakpoints EPA 2024 que la web.

• Se activa el sonido al superar ICA > 100 y se corta al bajar de 80.

• El silencio manual dura 10 minutos.

Endpoints HTTP

Ruta	Función
/silenciar?k=CLAVE	Silencia el zumbador durante 10 minutos (requiere la clave del dispositivo).
/status	Estado del dispositivo: conexión WiFi, ICA, alerta y silencio.
/alarma	Estado de la alarma (ICA, alerta, zumbando, silenciado).
/save (POST)	Guarda las credenciales WiFi y confirma si logró conectarse.
/ (portal)	Página de configuración; muestra el estado de la conexión WiFi.

Todos los endpoints de control envían encabezados CORS (Access-Control-Allow-Origin: *) para poder ser llamados desde la aplicación web.

15. Integración del sistema y control del zumbador

El flujo completo de una medición es:

• El Arduino Uno lee los sensores y el GPS y envía la trama de 9 campos al ESP32 por puerto serie.

• El ESP32 interpreta la trama, calcula el ICA y publica el registro en Supabase (incluida su IP).

• La aplicación web lee Supabase, muestra los datos y guarda la IP publicada por el ESP32.

• Al pulsar «Detener zumbador», la app llama a http://<ip>/silenciar?k=… y el ESP32 silencia el zumbador. No hace falta escribir ninguna IP: se toma sola desde la base de datos.

Requisitos de red

• El ESP32 y el dispositivo que usa la app deben estar en la misma red local para el control del zumbador.

• Si la estación está únicamente en modo punto de acceso, la IP del zumbador es siempre 192.168.4.1 (valor por defecto en la app).

Nota: la app se sirve por HTTP (por ejemplo con Live Server), por lo que la llamada al ESP32 por HTTP no presenta problemas de contenido mixto.

16. Correcciones de errores durante el desarrollo

Problema detectado	Causa	Solución aplicada
Los modales no se cerraban y aparecían encimados.	Reglas CSS .modal con display:grid/flex pisaban el atributo «hidden».	Se agregó [hidden]{display:none !important}.
El gráfico del histórico se veía colapsado en móvil.	Se creaba con el modal oculto (sin tamaño).	Se reordenó: abrir modal → crear gráfico → redimensionar.
El aviso «offline» desbordaba el borde en el celular.	Caja fuera del contenedor y sin márgenes.	Se envolvió el aviso en un .container.
Los paneles de Tendencia/GPS se salían de la tarjeta en móviles angostos.	El lienzo de Chart.js tiene ancho intrínseco de 300 px.	Grillas con minmax(0, …) y min-width:0 en .panel y .chart-wrap.
Umbrales sin respaldo normativo.	Rangos elegidos a criterio propio.	Se recalibraron con fuentes (OSHA/NIOSH, EPA, OMS/ICNIRP, NWS, ASHRAE).
El mapa mostraba «API key required» sobre fondo negro.	Los mosaicos de Carto requieren clave.	Se cambió a mosaicos de OpenStreetMap (sin clave).
La descarga JSON salía «entrecortada».	Pretty-print con muchos registros y MIME sin charset.	JSON sin indentación y MIME application/json;charset=utf-8.
El modal de descarga se desbordaba en Chrome Android al abrir los desplegables.	Grillas 1fr que no podían encogerse por el ancho intrínseco del <select>.	minmax(0,1fr), min-width:0, overflow-x:hidden y ancho del modal limitado al viewport.
El texto «JSON (datos crudos)» se cortaba en PC.	Modal demasiado angosto (420 px).	Se ensanchó el modal de descarga a 560 px.
La app enviaba datos a Supabase y recibía HTTP 400.	Se enviaba un campo «aqi» inexistente.	Se eliminó el campo del payload.
Las lecturas quedaban desalineadas (valores cruzados).	El Uno enviaba 8 campos y el ESP32 interpretaba 7.	Se unificó la trama en 9 campos y el parser correspondiente.
Llegaban coordenadas (0,0) a la base.	El GPS enviaba 0,0 al perder el fix.	El Uno conserva y reenvía la última coordenada válida.
El sketch del Arduino Uno quedó sobrescrito con el de la ESP32.	Reemplazo accidental del archivo.	Se restauró el firmware del Uno y se separaron ambos en carpetas propias.

17. Verificaciones y pruebas realizadas

Verificación	Resultado
Sintaxis del JavaScript (esprima).	Correcta (sin errores de análisis).
Coincidencia de los identificadores usados por app.js con el HTML.	Todos presentes.
Respuesta de la API REST de Supabase.	HTTP 200; columnas con los nombres esperados.
Escritura real a Supabase (POST desde el flujo).	HTTP 201 (registro creado correctamente).
Compilación del firmware del Arduino Uno (avr-g++).	Sin errores.

Verificación	Resultado
Compilación del firmware del ESP32 (xtensa-esp32-elf-g++).	Sin errores.
Parser NMEA del GPS (prueba automatizada).	Correcto (sentencias N/S/E/W y sin fix).
Renderizado del panel (navegador headless).	Sin errores de JavaScript; mapa y tarjetas correctos.
Columna «ip_dispositivo» en Supabase.	Operativa; el ESP32 publica su IP en cada envío.

Prueba pendiente en campo: validar la alarma sonora y el corte del zumbador durante una presentación real con la estación conectada a la red del lugar.

18. Cómo ejecutar y visualizar el sistema

Aplicación web con Live Server (Visual Studio Code)

• Abrir la carpeta App-Web2 en Visual Studio Code.

• Instalar la extensión «Live Server» y pulsar «Go Live».

• Se abrirá el panel en http://127.0.0.1:5500/.

• Desde un celular de la misma red: http://<IP-de-la-PC>:5500/ (la IP se obtiene con hostname -I).

Opción alternativa — servidor local con Python

```
python3 -m http.server 8000 --bind 0.0.0.0
```

• En la misma PC: http://127.0.0.1:8000/

• En un celular de la misma red Wi-Fi: http://<IP-de-la-PC>:8000/

Requisitos

• Conexión a internet (librerías por CDN y consulta a Supabase).

• Para el control del zumbador, estar en la misma red local que el ESP32.

19. Pendientes y recomendaciones

Tema	Recomendación
Publicación online	Hostear los 3 archivos (GitHub Pages, Netlify, Vercel o Cloudflare Pages) para una URL fija.
Seguridad Supabase	Verificar RLS con lectura pública y bloqueo de escritura salvo el rol del dispositivo.
Calibración	Calibrar el MQ-7 (R0) y validar el LDR frente a un luxómetro de referencia.
GPS	Usar una antena con buena vista del cielo para mejorar el tiempo de fix.
Alarma	Probar en una presentación real el disparo y el corte remoto del zumbador.
Fuente de alimentación	Usar una fuente capaz de sostener los picos de corriente del Wi-Fi.

Anexo A — Configuración y claves

El bloque CONFIG de app.js centraliza la configuración del panel:

```
const CONFIG = {
supabaseUrl: "https://lskvyvpjjziuclaatrtu.supabase.co",
supabaseKey: "sb_publishable_O9Havbh6tsAZEDA8VFWQzg_LmTmU4mM",
table: "mediciones",
historyLimit: 100,
pollMs: 15000,
staleWarnSeconds: 120,
staleOfflineSeconds: 300,
defaultLat: -26.82414,
defaultLng: -65.2226,
downloadRanges: { "100": 100, "500": 500, "1000": 1000, all: Infinity },
deviceKey: "tucu2026",
deviceIpDefault: "192.168.4.1",
muteMinutes: 10
};
```

Nota de seguridad: «supabaseKey» es una clave pública (anon) pensada para usarse en el cliente; la protección de los datos depende de las políticas RLS del servidor. La clave «deviceKey» es la que exige el ESP32 en el endpoint «/silenciar».

Anexo B — Exportación de datos (CSV / JSON)

CSV

El archivo CSV incluye una marca de orden de bytes (BOM) para correcta apertura en planillas, usa el punto y coma (;) como separador y contiene las columnas:

```
id;fecha_hora;pm25_ug_m3;co2_ppm;co_ppm;temperatura_c;humedad_pct;
presion_hpa;uv;luz_lux;latitud;longitud
```

El nombre del archivo sigue el patrón tucuman_respira_AAAA-MM-DD.csv.

JSON

El archivo JSON contiene el arreglo de registros tal como los devuelve la API, con todos los campos de cada medición (incluidos latitud, longitud e ip_dispositivo). El nombre sigue el patrón tucuman_respira_AAAA-MM-DD.json. Se genera sin indentación y con codificación UTF-8 para evitar cortes en archivos grandes.

Anexo C — Trama de comunicación Uno → ESP32

Comunicación serie a 9600 baudios, una línea de texto por medición, con 9 campos separados por comas:

Posición	Campo	Ejemplo
1	CO■ (ppm)	412
2	CO (ppm)	3.20
3	Temperatura (°C)	19.9

Posición	Campo	Ejemplo
4	Humedad (%)	71
5	Presión (hPa)	968
6	UV (índice)	0.0
7	Luz (lux)	120
8	Latitud (grados decimales)	-26.83972
9	Longitud (grados decimales)	-65.29328

El ESP32 procesa la trama con una lectura de formato tipo sscanf("%d,%f,%f,%f,%f,%f,%d,%f,%f") y descarta las tramas inválidas (contador de tramas inválidas).

Anexo D — Prompt para recrear el proyecto con opencode

A continuación se incluye un prompt ampliado, listo para copiar y pegar en opencode u otro asistente, con la información técnica necesaria para reconstruir el sistema (panel web y firmwares).

```
Actua como desarrollador full-stack senior en sistemas embebidos y frontend. Reconstruye de
forma
COMPLETA y FUNCIONAL el sistema "Tucuman Respira": una estacion de monitoreo de calidad del
aire
con un Arduino Uno + ESP32 que publica en Supabase y un panel web estatico que muestra los
datos.

=== PANEL WEB (estatico, 3 archivos) ===
- index.html, styles.css, app.js (sin backend, sin build).
- Lee Supabase (tabla mediciones) por la API REST con la clave anon.
- 8 sensores con umbrales EPA/OMS-ICNIRP/OSHA-NIOSH/ASHRAE; ICA (EPA) con PM2.5 y CO.
- Historico por sensor, grafico de tendencia, descarga CSV/JSON, mapa Leaflet con
OpenStreetMap.
- El mapa usa la ultima coordenada GPS valida si la lectura actual no tiene posicion.
- Boton "Detener zumbador" que toma la IP del dispositivo desde la columna ip_dispositivo
de
Supabase y llama a http:///silenciar?k=tucu2026 (fallback 192.168.4.1).

=== FIRMWARE ARDUINO UNO (sketch_oct17a) ===
- BME280 (I2C A4/A5), MH-Z19C (SoftwareSerial 10/11), MQ-7 (A0), LDR (A1/D12), ML8511 (A2),
GPS NEO-6M (SoftwareSerial 7/8), ESP32 (SoftwareSerial 2/3).
- Parsea $GPGGA/$GNGGA a grados decimales; conserva y reenvia la ultima coordenada valida.
- Envia cada 3 s una trama de 9 campos: co2,co,temp,hum,pres,uv,luz,lat,lon.

=== FIRMWARE ESP32 (sketch_oct17b) ===
- Conexion WiFi STA con reconexion + AP permanente Tucuman-Respira-Config (192.168.4.1).
- POST a Supabase con latitud, longitud e ip_dispositivo (= WiFi.localIP()).
- ICA EPA 2024 local; zumbador en GPIO 23 (tone 2000): ON >100, OFF <80, silencio 10 min.
- Endpoints /silenciar?k=tucu2026, /status, /alarma, /save; CORS abierto.
- Portal de configuracion que confirma el estado de la conexion WiFi.

=== SUPABASE ===
- Tabla mediciones (id, created_at, co2, co, pm25, temperatura, humedad, presion, uv, luz,
latitud, longitud, ip_dispositivo).
```